

iOS Location Spoofer — DIY аналог iMyFone AnyTo

Цель проекта

Создать desktop-приложение для подмены геолокации на iPhone двумя способами:

1. **DVT-режим** — системная подмена GPS через Developer Disk Image для целей разработки и тестирования геолокационных приложений (аналог функции Simulate Location в Xcode, но с GUI и расширенными возможностями)
2. **BLE-режим** — эмуляция Bluetooth-аксессуара внешнего GPS для игр, поддерживающих внешние GPS-устройства (Pokémon GO, Monster Hunter Now)

Scope MVP: одна страна (Польша), локальный радиус действия до ~50 км от реальной позиции пользователя. Межстрановое перемещение, VPN-маршрутизация и cooldown-менеджмент для дальних телепортов — out of scope.

Основные сценарии использования:

- QA-тестирование геолокационных функций собственных приложений
- Защита приватности (контроль того, какие координаты видят установленные приложения)
- Гейминг с поддержкой внешних GPS-аксессуаров
- Разработка и отладка location-based сервисов

Целевые платформы

- **Основная разработка:** Ubuntu 24.04 (BlueZ для BLE peripheral)
- **Production-сборки:** Windows 10/11, Linux (Ubuntu/Debian/Fedora)
- macOS — не обязательно на этапе MVP

Технологический стек

Backend (Python 3.11+)

- `pymobiledevice3` — взаимодействие с iOS (DVT, tunneled, lockdownd)

- `bleak` — кроссплатформенный BLE-клиент (для тестов)
- `dbus-next` + прямая работа с BlueZ — для BLE peripheral mode на Linux
- `cryptography` — AES-CTR для PGP-аутентификации
- `fastapi` + `uvicorn` — локальный HTTP/WebSocket API между бэком и фронтом
- `pydantic` — модели данных
- `dnslib` — кастомный DNS-сервер для дальних телепортов в играх

Frontend (Tauri + React + TypeScript)

- Tauri 2.x — лёгкая нативная обёртка
- React 18 + TypeScript
- Mapbox GL JS или MapLibre — интерактивная карта
- Zustand — стейт-менеджмент
- Tailwind CSS — стилизация
- `shadcn/ui` — компоненты

Архитектура

```

project/
├── backend/                                # Python-сервис
│   ├── main.py                            # FastAPI entry point
│   ├── core/
│   │   ├── ios_manager.py                 # Управление подключением iPhone
│   │   ├── dvt_spoofers.py                # DVT/simulate-location обёртка
│   │   ├── ble_pgp.py                     # BLE-эмуляция Pokémon Go Plus
│   │   ├── pgp_crypto.py                   # AES-CTR аутентификация PGP
│   │   ├── pgp_key_store.py                # Хранение сертификационных ключей
│   │   ├── dns_proxy.py                   # DNS-прокси для дальних прыжков
│   │   ├── bt_pan.py                       # Управление Bluetooth PAN (раздача интернета по B
│   │   ├── preflight.py                    # Pre-flight проверка окружения перед стартом
│   │   ├── companion_sync.py               # Синхронизация с iOS companion app
│   │   ├── real_location.py                # Определение реальной локации устройства (где iPh
│   │   └── movement.py                     # Интерполяция маршрутов
│   └── api/
│       ├── devices.py                      # /api/devices/* эндпоинты
│       ├── location.py                     # /api/location/* эндпоинты
│       └── preflight.py                     # /api/preflight/* эндпоинты

```

```

|   |   └─ ws.py                # WebSocket для real-time апдейтов
|   └─ models/                  # Pydantic-модели
|   └─ tests/
└─ frontend/                    # Tauri + React
    └─ src/
        └─ components/
            └─ Map.tsx           # Карта Mapbox
            └─ ModeSelector.tsx
            └─ DeviceList.tsx
            └─ Joystick.tsx
            └─ RouteBuilder.tsx
            └─ LocateMeButton.tsx # Кнопка "найти реальную позицию" + цент
            └─ PreflightChecklist.tsx # Проверка окружения перед запуском
            └─ DriftMonitor.tsx    # Мониторинг рассинхрона BLE↔iOS
            └─ SpeedControl.tsx
        └─ hooks/
            └─ useDevice.ts
            └─ useLocation.ts
            └─ usePreflight.ts
            └─ useWebSocket.ts
        └─ store/                # Zustand
        └─ api/                  # Клиент бэкенда
    └─ src-tauri/                # Rust-обвязка Tauri
└─ ios-companion/               # iOS sideload-приложение (Swift)
    └─ Sources/
        └─ App.swift
        └─ LocationManager.swift # CLLocationManager + background mode
        └─ MotionDetector.swift  # CMMotionManager — детект стационарности
        └─ DriftDetector.swift   # Сравнение системной локации с целевой
        └─ BTSyncService.swift   # BLE-канал связи с десктоп-приложением
        └─ AirplaneModeAdvisor.swift # Полноэкранные алерты о необходимости вклю
        └─ FocusModeHelper.swift # Интеграция с iOS Focus modes
    └─ Resources/
    └─ README.md                 # Инструкция по сборке и sideload через AltStore/S
└─ tools/
    └─ pgp_key_extractor/        # Утилита для извлечения ключа из реального PGP
    └─ mac_randomizer/           # Скрипт смены MAC Wi-Fi адаптера на ПК (для хотсп
└─ docs/
    └─ location-sources.md       # Как iOS определяет локацию и как это изолировать
    └─ connection-schemes.md     # Схемы подключения интернета (BT PAN, VPN, хотсп
    └─ troubleshooting.md        # Решение типичных проблем

```

**Ключевая техническая проблема: изоляция источников
локации**

iOS определяет геолокацию из нескольких независимых источников:

GPS/GLONASS/Galileo, Wi-Fi (по базе MAC-адресов роутеров Apple Location Services), сотовые вышки и iBeacon. BLE-режим подменяет координаты только для приложений, читающих с BLE-аксессуара, но iOS параллельно продолжает определять реальную локацию через остальные каналы. Игры с серверной валидацией сравнивают BLE-координаты с системной локацией и при расхождении >5–8 км считают это спуфингом.

DVT-режим эту проблему не имеет (там подменяется системная локация целиком), но имеет требование USB/Wi-Fi подключения и не работает с приложениями, валидирующими координаты через дополнительные каналы.

Источники локации iOS и стратегия их изоляции

Источник	Частота	Точность	Как изолировать
GPS L1	1575 МГц	±5–20 м	Авиарежим, клетка Фарадея, программно через Location Services
Wi-Fi (ALS)	2.4/5 ГГц	±10–50 м в городе	Wi-Fi off, либо подключение к роутеру с неизвестным Apple BSSID
Сотовые вышки	700–2600 МГц	±100–3000 м	Авиарежим, либо отключение Cellular Data
iBeacon/BLE окружения	2.4 ГГц	±1–10 м	Сложно, обычно несущественно

Поддерживаемые схемы подключения интернета

Основная схема: Cellular + DNS-блок Apple Location

Подходит для всех локальных сценариев (BLE-координаты в пределах ~50 км от реальной позиции пользователя).

- Cellular Data включена, Wi-Fi выключен
- Локальный DNS-сервер блокирует `gs-loc.apple.com` и связанные хосты Apple Location Services
- iPhone не может уточнить локацию через Wi-Fi → точность падает до cellular tower triangulation (±500–3000 м)
- `horizontalAccuracy` высокая → антифрод-окно расширено до 20–30 км

- Скорость интернета — полная (4G/5G)
- Setup для пользователя: минимальный (только переключить DNS в Wi-Fi-настройках iPhone на время сессии, либо установить .mobileconfig профиль один раз)

Оptionальная схема: Авиарежим + Bluetooth PAN

Нужна только если пользователь хочет полную изоляцию (например, тесты с дальними координатами, отладка). Out of scope для MVP, можно добавить позже.

Детектор стационарности

В движении iOS не успевает накопить достаточно сигналов для уверенного фикса локации. Стационарное положение (>30 сек без движения) — основной триггер рассинхрона. Companion-app на iPhone должен:

- Читать акселерометр через `CMMotionManager` (не требует location permission)
- При обнаружении стационарности >30 сек — предупреждать пользователя
- Логировать события рассинхрона для последующего анализа

Влияние `horizontalAccuracy` на допустимое расхождение

Игры с серверной валидацией координат сравнивают BLE-координаты с системной локацией iOS не по жёсткому порогу, а с учётом поля `horizontalAccuracy` (погрешности), которое iOS отдаёт вместе с координатами. Логика примерно такая:

```
threshold = max(BASE_THRESHOLD, ios_horizontalAccuracy * FACTOR)
```

Когда iOS уверена в локации (точный GPS-фикс, $\pm 5\text{--}10$ м) — порог расхождения минимальный (~ 8 км). Когда iOS не уверена ($\pm 300\text{--}1000$ м в помещении, без GPS, на одиночном Wi-Fi BSSID) — порог расширяется до 20–30 км.

Использование `Precise Location toggle`:

Начиная с iOS 14, пользователь может выключить «Precise Location» для конкретного приложения в Settings → Privacy & Security → Location Services → [App]. При выключенном Precise Location iOS возвращает координаты с искусственно завышенной погрешностью ($\pm 1\text{--}5$ км). Это легитимная системная функция, не требует джейлбрейка или sideload.

Добавляется в Pre-flight Checklist:

- Precise Location для целевого приложения **выключен** (расширяет допустимый

радиус прыжков с 8 км до 20–30 км)

Companion-app может детектить состояние Precise Location через

`CLLocationManager.accuracyAuthorization` и подсказывать пользователю переключить его при необходимости. Если приложение требует Precise Location для своей работы — Pre-flight выдаёт жёлтое предупреждение вместо красной блокировки.

Режимы перемещения (общие для DVT и BLE)

1. **Teleport** — мгновенный прыжок в точку
2. **Two-spot route** — движение между двумя точками с заданной скоростью
3. **Multi-spot route** — маршрут по N точкам
4. **Joystick** — ручное управление направлением и скоростью в реальном времени
5. **GPX import/export** — загрузка готовых маршрутов

Реализм движения

Основной режим — **движение по автомобильным маршрутам** (driving mode) от точки А до точки Б по реальной дорожной сети.

- Использование Mapbox Directions API с профилем `driving-traffic` (или локальный OSRM с дампом OSM Польши как бесплатная альтернатива)
- Скоростной профиль по типам дорог (motorway: 130, primary: 80, residential: 40 и т.д.)
- Плавное ускорение (2.5 м/с^2) и торможение (3.0 м/с^2)
- Замедление перед поворотами
- Случайные остановки на светофорах (40% вероятность на `traffic_signal` точках, 20–60 сек)
- Эмуляция пробок на сегментах с congestion data
- Замедление при подъезде к финишу (имитация парковки)
- Координатное дрожание $\pm 2 \text{ м}$ (имитация погрешности автомобильного GPS)
- Периодические «дропы» сигнала на 1–3 сек (туннели, мосты)
- Пешеходный режим (walking) — опционально, как вторичный для финальных метров маршрута

Управление подключением

- Автодетект подключённого iPhone по USB
- Поддержка Wi-Fi pairing после первого USB-подключения
- Trust dialog handling
- iOS 17+ tunneld setup (требуется sudo на Linux/Mac, admin на Windows)
- Корректное закрытие сессии при отключении

BLE-режим специфика

- Эмуляция PGP с правильным advertisement name
- Реализация трёх GATT-сервисов PGP
- AES-CTR аутентификация (reconnection protocol + first-pair protocol)
- Загрузка сертификационного ключа из локального хранилища (пользователь предоставляет сам)
- **DNS-прокси для модификации серверных ответов при перемещениях >20 км** — нужен только если сервер игры выполняет дополнительную валидацию координат относительно сетевого IP

Pre-flight Checklist (обязательный компонент)

Перед запуском любой эмуляции в BLE-режиме приложение показывает чек-лист состояния окружения. Запуск блокируется, пока все критические пункты не зелёные.

Минимальный набор проверок (для MVP-сценария Cellular + DNS-блок):

- iPhone подключён по BT и связан с десктоп-приложением
- Cellular Data включена на iPhone
- Wi-Fi выключен на iPhone (или подключение только к доверенной сети без Apple Location lookup)
- Локальный DNS-сервер запущен и блокирует Apple Location Services
- iPhone использует этот DNS (профиль установлен либо ручная настройка проверена)
- Location Services включены для целевого приложения (нужно для работы аксессуара)

- **Precise Location выключен** для целевого приложения (расширяет допустимое окно расхождения)
- Companion-аpp на iPhone запущен и связан с десктоп-приложением

Каждый пункт должен иметь:

- Статус (зелёный/жёлтый/красный)
- Текстовое объяснение проблемы
- Кнопку «Как исправить» с пошаговой инструкцией
- Кнопку «Открыть настройки» через iOS URL Scheme где возможно

Drift Monitor (мониторинг рассинхрона)

Реал-тайм компонент в GUI, который показывает:

- Текущие BLE-координаты (что отдаёт эмулятор)
- Системные координаты iOS (что получает companion-app от Core Location)
- Дистанцию между ними
- Цветовой индикатор: зелёный (<2 км), жёлтый (2–5 км), красный (>5 км)
- При красном статусе — автоматическая пауза эмуляции и алерт

iOS Companion App (отдельный sideload-проект)

Нативное Swift-приложение, устанавливаемое на iPhone через AltStore/Sideloadly.

Функции:

- BLE-связь с десктоп-приложением (свой GATT-сервис, отдельно от PGP-эмулятора)
- Чтение системной локации через `CLLocationManager` (background mode)
- Передача системной локации на десктоп для Drift Monitor
- Чтение акселерометра через `CMMotionManager` для детекта стационарности
- Полноэкранные алерты при необходимости включить авиарежим
- Интеграция с iOS Focus modes для автоматизации настроек
- Local notifications при разрыве BLE-связи
- Inline-инструкции и диплинки в системные настройки

Методы установки Companion-аpp на iPhone

В `ios-companion/README.md` должна быть таблица «iOS version → recommended install method» и инструкции для каждого варианта.

iOS на устройстве	Рекомендуемый метод	Apple ID нужен	Срок действия установки
≤ 16.6.1	TrollStore	Нет	Постоянно
17.0 (некоторые билды)	TrollStore через kfd exploit, иначе SideStore	Только для SideStore	Постоянно для TrollStore / 7 дней для SideStore
17.1 – 17.6	SideStore	Да (бесплатный)	7 дней, auto-renew по Wi-Fi
17.7+ / 18+	SideStore (с актуальной версией)	Да (бесплатный)	7 дней, auto-renew

TrollStore (предпочтительный путь для совместимых iOS):

- Использует CoreTrust-баг для постоянной установки .ipa без подписи
- Не требует Apple ID вообще
- Установка через ios.cfw.guide/installing-trollstore/
- После установки достаточно один раз скачать .ipa нашего companion-аpp и установить через TrollStore — работает бессрочно
- Не путать с jailbreak: TrollStore — это просто permasigned-инструмент, не даёт root

SideStore (когда TrollStore не подходит):

- Требуется бесплатный Apple ID (не платный \$99 developer account)
- Сертификаты обновляются автоматически по Wi-Fi через JIT-сервер
- Лимит — 3 приложения на один Apple ID
- Установка: sidestore.io

AltStore (классический путь):

- Альтернатива SideStore, требует AltServer на ПК
- ПК должен периодически быть в одной Wi-Fi сети с iPhone для re-signing

- Тот же бесплатный Apple ID

Подготовка Apple ID для sideload

Рекомендация для пользователей: завести **отдельный «технический» Apple ID** специально для sideload, не связанный с основным iCloud/банковскими приложениями.

Шаги:

1. Создать новый email (можно через одноразовый сервис типа SimpleLogin/AnonAddy с алиасом)
2. На appleid.apple.com или прямо на устройстве: Settings → Sign In → Create New Apple ID
3. Имя/фамилия — любые
4. Дата рождения — любая (старше 13 лет)
5. Адрес — любой реальный почтовый индекс той страны где регистрируешься (валидируется по формату, не по реальности)
6. **Карту НЕ привязывать** — пропустить шаг (для бесплатного Apple ID не требуется)
7. Если Apple блокирует создание — использовать VPN (US/UK обычно проходят), либо создать с другого устройства

Этот Apple ID используется ТОЛЬКО для sideload — не входить с ним в iCloud, App Store, iMessage. Sign-in только в SideStore/AltStore приложениях.

First-Run Wizard (десктоп-приложение)

Визуальный мастер первого запуска, проводящий пользователя по подготовке окружения:

1. Детектит подключённый iPhone, показывает модель и iOS-версию
2. По iOS-версии рекомендует метод установки (TrollStore / SideStore / AltStore)
3. Ссылка на пошаговую инструкцию под выбранный метод
4. Генерирует и предлагает установить .mobileconfig (DNS-блокировка Apple Location)
5. Помогает пройти Trust This Computer
6. Проверяет успешность pairing
7. Подсказывает однократную перезагрузку iPhone

8. Финальная валидация: все компоненты на месте, можно начинать сессию

Session Wizard (десктоп-приложение)

Короткий чек-лист перед каждой сессией:

- Авто-детект подключения iPhone (USB или Wi-Fi pairing)
- Авто-детект состояния Cellular / Wi-Fi на iPhone (через companion-app)
- Авто-детект что DNS-блок активен
- Авто-детект что Precise Location выключен для целевого приложения
- Авто-детект что companion-app запущен
- Кнопки «Fix It» с диплинками в нужный раздел iOS Settings для каждой проблемы
- Блокировка Start пока все пункты не зелёные

Bluetooth PAN Manager

Модуль на десктопе, который автоматизирует раздачу интернета по BT:

- На Linux: настройка через NetworkManager + bnetp
- На Windows: через Bluetooth PAN API (ICS)
- Автоматическое поднятие соединения при старте сессии
- Мониторинг пропускной способности
- Опциональное проксирование через VPN перед раздачей

Distribution & Bundling

Финальное приложение должно распространяться как **standalone executable** на каждую платформу, без необходимости пользователю устанавливать какие-либо зависимости (Python, Node.js, iTunes, runtime-библиотеки и т.д.).

Архитектура сборки

```
Tauri (frontend bundle)
├─ resources/
│   └─ backend (PyInstaller-frozen)
│       ├── Python 3.11 runtime
│       ├── pymobiledevice3 + libimobiledevice (нативные либы)
│       └── cryptography + OpenSSL
```

```
└─ bleak / dbus-next
└─ fastapi + uvicorn
└─ всё остальное из requirements.txt
```

Tauri запускает backend как sidcar-процесс, общение через локальный HTTP/WebSocket на 127.0.0.1.

Что НЕ требует установки пользователем

Зависимость	Где берётся
Python interpreter	Внутри backend executable (PyInstaller)
libimobiledevice	Внутри rymobiledevice3 wheels
OpenSSL	Внутри cryptography wheel
Node.js / npm	Не нужен в production (Tauri компилирует фронтенд)
BlueZ (Linux)	Системный, поставляется со всеми дистрибутивами
Bluetooth stack (Windows)	Системный, Microsoft
iTunes / Apple Mobile Device Service	Не требуется — современный rymobiledevice3 работает напрямую через usbmux/WinUSB

Что может потребоваться при первом подключении iPhone

- **Windows:** автоматическая установка USB-драйвера iPhone через Windows Update при первом подключении (без участия пользователя)
- **Linux:** пользователь возможно должен быть в группе `plugdev` для доступа к USB (udev-правило ставится postinst-скриптом в `.deb`)
- **iOS-сторона:** один раз “Trust This Computer” при USB-подключении (родная iOS-функция, не установка)

Сборочные команды

Windows:

```
# Сборка backend
cd backend
pyinstaller --onefile --name backend main.py
```

```
# Сборка frontend + упаковка
cd ../frontend
npm run tauri build
# Результат: src-tauri/target/release/bundle/msi/PokemonSpoofers.msi
# или nsis installer
```

Linux:

```
cd backend
pyinstaller --onefile --name backend main.py
cd ../frontend
npm run tauri build
# Результат: .deb, .AppImage, .rpm в зависимости от настроек tauri.conf.json
```

Размер финального бандла

Ожидаемый размер: 80–150 МБ per platform. Это нормально для приложений уровня AnyTo и подобных.

CI/CD сборка

GitHub Actions workflow для автоматических сборок на push tag:

- matrix: ubuntu-latest, windows-latest
- artifacts: .exe / .msi / .deb / .AppImage
- release publish на GitHub Releases

Error Recovery & Session Resilience

Сессия может прерваться по множеству причин: iPhone отключился от USB, BLE-разрыв, сел аккумулятор, rumbleddevice3 кинул exception, игра свернулась. Приложение должно корректно обрабатывать все эти случаи без потери состояния маршрута.

Модуль `core/recovery.py`

- **State persistence:** текущая позиция, активный маршрут, прогресс, выбранный режим — сохраняются на диск каждые 5 сек в SQLite
- **Auto-reconnect:** при разрыве USB/BLE — попытки переподключения с exponential backoff (1с, 2с, 5с, 10с, 30с, потом каждые 60с)
- **Resume from last known position:** после восстановления связи маршрут

продолжается с точки разрыва, не с начала

- **Graceful degradation:** если DVT упал — попытка переключиться на BLE и наоборот (там где применимо)
- **Crash recovery:** при следующем запуске после краша — окно “Восстановить прерванную сессию?” с превью маршрута

Типичные исключения и их обработка

Exception	Действие
<code>MuxException</code> (потеря USB-связи)	Auto-reconnect + UI notification
<code>LockdownError</code> (Trust revoked)	Алерт “Re-trust this computer” с диплинком
<code>DDIMountError</code> (не смонтировался DDI)	Retry с downloading свежего DDI
<code>BLEDisconnect</code>	Auto-reconnect, если игра ещё ждёт аксессуара
<code>RemoteXPCErrors</code> (iOS 17+ tunneld)	Restart tunnel, retry
Unhandled exception	Сохранить state, показать crash dialog с кнопкой “Generate diagnostic report”

Logging & Diagnostics

Структурированное логирование — критическая инфраструктура для поддержки проекта.

Backend logging

- Библиотека: `structlog` (структурированные JSON-логи)
- Уровни: `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`
- Ротация: `logging.handlers.TimedRotatingFileHandler` — по одному файлу в день, хранение 14 дней
- Расположение логов:

- Linux: `~/.local/share/pokemon-spoofers/logs/`
- Windows: `%APPDATA%/PokemonSpoofers/logs/`
- macOS: `~/Library/Application Support/PokemonSpoofers/logs/`
- Категории: `device, ble, dvt, routing, network, ui, api`

Frontend logging

- Tauri логи в тот же каталог через `tauri-plugin-log`
- Уровень устанавливается из настроек приложения

Что НЕ логировать

- PGP сертификационные ключи (даже хеши)
- Apple ID учётные данные
- Содержимое `.mobileconfig` с DNS-настройками (только факт установки)
- Точные координаты пользователя в production-логах (только город / area, опционально)

Diagnostic Report

Кнопка в Settings → "Generate Diagnostic Report":

- Собирает: версия приложения, OS, последние 500 строк логов, текущий конфиг (без секретов), результаты pre-flight, состояние подключения
- Упаковывает в zip с timestamp
- Открывает Finder/Explorer с готовым файлом
- Юзер может приложить к issue в GitHub или отправить разработчику

Dev mode

Hidden hotkey `Ctrl+Shift+D` открывает dev panel:

- Live-лог в окне приложения
- Метрики: latency BLE-пакетов, jitter координат, drift по времени
- Сырые данные от `rumobiledevice3`
- Manual override команды для тестирования

Auto-Updates

Критично из-за того, что Apple ломает internal APIs в каждом мажорном релизе iOS — приложение должно уметь быстро доставлять патчи.

Tauri Updater

- Использовать встроенный `@tauri-apps/plugin-updater`
- Update manifest на GitHub Releases или собственном сервере
- Подпись апдейтов через `tauri signer` (ed25519 keypair, публичный ключ зашит в приложение)
- Проверка обновлений при запуске и раз в 24 часа в фоне

UX обновлений

- Non-intrusive notification "Доступна новая версия (1.2.3)"
- Changelog отображается перед установкой
- Кнопки "Install Now" / "Remind Me Later" / "Skip This Version"
- Critical updates (помечены в manifest) — настойчивее, но не форсятся
- Автоматическая установка только если пользователь выбрал в Settings

Backend-only updates

Если изменился только Python-бэкенд (например, обновился `rumobiledevice3` под новый iOS), можно доставлять только новый `sidecar binary` без переустановки всего приложения. Уменьшает размер апдейтов с 120 МБ до 30 МБ.

Testing Strategy

Без тестов проект через 6 месяцев становится `untouchable`. Минимальное покрытие критических компонентов обязательно.

Unit-тесты (pytest)

- **PGP криптография**: тестовые векторы из pgreми для AES-CTR + handshake
- **Movement interpolator**: проверка плавности, корректности скорости, дрожания
- **Route parser**: парсинг GeoJSON от Mapbox, GPX import/export

- **DNS logic:** правила блокировки, режимы работы
- **Recovery logic:** моки разрывов соединения, проверка state restoration

Integration-тесты

- Требуют подключённый iPhone (отмечаются `@pytest.mark.requires_device`)
- Запускаются вручную и на dedicated CI-машине с подключённым iPhone
- Покрывание: pairing flow, DVT mount, simulate-location, BLE connect

Frontend-тесты

- Vitest для компонентов
- Playwright для E2E (моки бэкенда)
- Snapshot-тесты для критических UI-состояний (pre-flight green/yellow/red)

Mock-инфраструктура

Создать `tests/mocks/` :

- `MockiPhone` — fake-устройство, отвечающее на `rumobiledevice3` запросы
- `MockGATTClient` — эмуляция клиента BLE для тестов `peripheral mode`
- `MockMapbox` — фиксированные ответы `Directions API`

Это позволяет 80% тестов гонять без реального железа.

CI matrix

GitHub Actions:

- Lint (ruff, mypy, eslint, tsc)
- Unit tests на Ubuntu и Windows
- Integration tests — manual trigger only (нужно железо)
- Build artifacts только при успешных тестах

Итерация 1: Минимальный DVT-режим

- Подключение iPhone по USB через `rumobiledevice3`
- Mount Developer Disk Image (iOS до 16)

- Простой CLI: `spoof --lat X --lon Y`
- Тест на реальном устройстве

Итерация 2: iOS 17+ поддержка

- tunneld через RemoteXPC
- Personalized DDI
- Обработка прав (sudo/admin)

Итерация 3: Базовый GUI

- Tauri-каркас + карта
- Teleport-режим из UI
- WebSocket-связь с бэкендом
- Список устройств

Итерация 4: Маршруты и движение

- Two-spot, Multi-spot
- Интерполятор с настраиваемой скоростью
- Joystick

Итерация 5: BLE-режим

- BlueZ peripheral на Linux
- GATT-сервисы PGP
- AES-CTR аутентификация
- Загрузчик ключей
- Switch UI: DVT ↔ BLE

Итерация 6: iOS Companion App

- Swift-проект `ios-companion/`
- CLLocationManager + background mode для чтения системной локации
- CMMotionManager для детекта стационарности

- BLE-канал связи с десктоп-приложением (свой GATT-сервис)
- Базовый UI: статус подключения, текущая системная локация, алерты
- Инструкция по sideload через AltStore/Sideloadly
- Тестирование связки десктоп ↔ companion на реальном устройстве

Итерация 7: Pre-flight Checklist и Drift Monitor

- Backend: модуль `preflight.py` с набором проверок
- API эндпоинты `/api/preflight/check`, `/api/preflight/status`
- Frontend компонент `PreflightChecklist.tsx` с цветовой индикацией
- Компонент `DriftMonitor.tsx` с real-time сравнением координат
- Блокировка кнопки «Start» при незелёных критических пунктах
- Пошаговые инструкции «Как исправить» для каждой проверки
- iOS URL Schemes для прямого открытия системных настроек

Итерация 8 (опционально): Bluetooth PAN Manager

Out of scope для MVP. Реализовать только если потребуется выходить за пределы локального радиуса ~50 км либо для тестовых сценариев с полной изоляцией.

- Модуль `bt_pan.py` для автоматизации раздачи интернета по BT
- Linux: NetworkManager + bnetp
- Windows: Bluetooth PAN через ICS
- UI: добавление BT PAN как альтернативной схемы подключения в pre-flight

Итерация 9: DNS-прокси и production polish

- DNS-сервер для специфических игровых сценариев
- GPX импорт/экспорт
- Сохранение избранных локаций
- Логирование, error handling
- Сборка установщиков (Tauri bundler) для Windows и Linux
- Документация в `docs/`: location-sources, connection-schemes, troubleshooting

Ограничения и предупреждения

- **Сертификационные ключи PGP не входят в проект** — пользователь должен предоставить ключ из своего реального PGP/Gotcha. В репозитории — только утилита для извлечения и инструкция
- **Apple ломает DVT API в каждом мажорном релизе iOS** — закладывать обновление rymobiledevice3 как зависимость
- **Никаких ключей, токенов, прокси-серверов в репозитории**
- Проект предназначен для разработки, тестирования и личного использования. README должен содержать дисклеймер о соблюдении применимого законодательства и условий использования сторонних сервисов

Что НЕ делать

- Не использовать localStorage в Tauri-фронте (использовать Tauri store API)
- Не хардкодить пути устройств — всё через rymobiledevice3 API
- Не использовать Selenium или браузерную автоматизацию — это нативное приложение
- Не публиковать чужие сертификационные ключи

Старт работы

Начни с **Итерации 1**: настрой Python-окружение, установи rymobiledevice3, напиши минимальный CLI-скрипт `backend/cli_spoof.py`, который подключается к iPhone по USB и подменяет координаты. Покажи мне результат перед переходом к следующей итерации.

Перед началом любой итерации — опиши, что планируешь сделать, и подожди подтверждения.